Sparse Matrix-Vector Multiplication Investigation

Matteo Morellini - 268427
University of Trento
github.com/MatteoMorellini/GPU-Computing-Deliverable-1
matteo.morellini@studenti.unitn.it

***Abstract*—Sparse matrix-vector multiplication is strongly affected by memory access irregularity and load imbalance. This work compares five CSR-based SpMV implementations on an NVIDIA A30 across ten SuiteSparse matrices: CSR-Scalar, CSR-Vector, CSR-Adaptive, CSR-Partial-Overlap, and cuSPARSE. The results show that no custom kernel is uniformly optimal: performance depends mainly on row-length distribution, memory coalescing, and long-row handling. cuSPARSE is the most robust baseline, while CSR-Adaptive is the strongest custom implementation.**

I. Methodology

The absence of a universally competitive format has long been recognized in the SpMV literature: optimal performance is highly dependent on both the distribution of non-zero elements within a matrix and the storage format in which they are represented [2]. As a result, a broad range of approaches has been proposed over the years.

A. Motivations to Exclude Non-CSR Formats

The irregular distribution of non-zero elements across sparse matrices causes random accesses into the dense vector $\mathbf{x}$, which in turn leads to uncoalesced memory transactions and significant memory access overhead. To mitigate this, domain-specific formats exploit the structural regularity that characterizes matrices from particular fields: graph and web matrices tend to be highly irregular, while matrices from theoretical chemistry often exhibit a block structure imposed by physical and chemical constraints.

Formats such as DIA, BCSR, and ELL [2] were designed with this structure in mind. The downside, however, is their domain specificity: they fail to generalize uniformly across matrix classes. Because of its broad applicability, CSR has become the de facto standard compression format. Converting an existing CSR matrix into one of these specialized formats incurs both runtime and memory overheads [3]. While such overhead can be amortized over many SpMV iterations, on a single computation the conversion time alone vastly exceeds the cost of the multiplication itself [4]. This paper therefore restricts its scope to kernels that operate directly on the CSR format.

B. Variations of CSR Tested

Recall that in CSR format, a sparse matrix A of size $m \times n$ with *nnz* non-zeros is represented by three arrays: `values` and `col_idx` of length *nnz*, and `row_ptr` of length $m+1$, where `row_ptr`$[i]$ stores the index in `values` of the first non-zero element of row i. The sequential SpMV kernel follows directly:

For basic CSR-based GPU kernels, the memory bandwidth-bound nature of SpMV is compounded by the inability to reuse elements of the dense vector and by potential load imbalance on irregular matrices [3]. Subsequent algorithms have therefore sought to preserve the advantages of CSR while mitigating its weaknesses. Two broad strategies appear in the literature:

1) Transform the matrix into a modified storage format to improve GPU utilization (e.g., CSR5). Such approaches share the conversion overheads discussed above.
2) Improve the SpMV kernel itself to fully exploit GPU hardware while leaving the CSR structure intact.

The second strategy is the one adopted here. Specifically, the following algorithms are evaluated.

CSR-Scalar [1] assigns one thread per row. It serves as the weakest baseline because, on top of the well-known weaknesses of CSR, it (i) accesses elements that are m positions apart, so memory accesses are never coalesced, and (ii) is susceptible to severe load imbalance when rows differ greatly in the number of non-zero elements.

CSR-Vector [1] assigns one warp per row. Threads within the same warp always access contiguous elements, improving effective bandwidth. However, rows with few non-zeros leave many threads idle within the warp. Formally, if a row contains $k \ll W$ non-zeros, where W is the warp size, then the warp utilization is at most k/W, leading to potential load imbalance within thread blocks.

CSR-Adaptive [4] was designed to address the cases where both previous algorithms degrade. Contiguous rows are grouped dynamically until their combined non-zero count fills at most one thread block, and a single block is then assigned to each such group. As explained in Algorithm 1, two execution paths are selected at runtime based on the number of merged rows:

- **CSR-Stream** is used when multiple rows fit within one block. Each thread loads its assigned non-zero element into shared memory, and a single thread per row performs the reduction. This directly addresses the idle-thread problem of CSR-Vector on short rows.
- A block-level variant of CSR-Vector is used when a single row contains more non-zeros than threads in a block, dedicating the entire block to that row.

The row-block partitioning required by CSR-Adaptive is generated on the CPU side before kernel launch. Its complexity

Algorithm 1 CSR-Adaptive

CPU: partition rows into blocks s.t. nnz/block $\leq$ BLOCKSIZE $O(m)$
for each block g **in parallel do**
  if |rows in g| $\leq$ SMALLVALUE **then**
    CSR-Stream: stream into shared mem; scalar reduction per row
  else
    CSR-Vector: one warp per row; parallel reduction
  end if
end for

Algorithm 2 Two-stage pipeline in CSR-Partial-Overlap

for each batch b **do**
  Produce: async_copy($vals_b, smem[b \bmod 2]$)
  Consume: compute $smem[(b-1) \bmod 2]$ after copy
end for

is $O(m)$, far cheaper than converting CSR into an alternative format.

CSR-Partial-Overlap [5] exploits the asynchronous memory copy instruction introduced by the NVIDIA Ampere architecture. The `memcpy_async` instruction transfers data directly from global to shared memory, bypassing L1 cache and freeing all thread resources during the transfer. This enables the two-stage pipeline shown in Algorithm 2.

Load balancing is achieved through a dynamic batch partition that ensures no row is split across two batches, thereby eliminating the partial-sum fixup step that would otherwise require additional global memory accesses. Thread utilization within each batch is further improved by dynamically selecting the number of threads assigned to each row based on the average non-zero count of that batch.

cuSPARSE, from NVIDIA, is included as an optimized baseline.

C. Experimental Setup

All GPU outputs are validated against a sequential CPU CSR implementation using a relative tolerance of $10^{-3}\max(1, |y_i^{cpu}|)$. Kernel time is measured with CUDA events over 100 repetitions after 5 warmup runs, excluding file parsing, format conversion, and host-to-device transfer. Throughput is reported as $2 \cdot nnz/t$ GFLOP/s. Experiments run on an NVIDIA A30 using Float32 inputs generated with a fixed random seed.

II. DATASET

Ten matrices from the SuiteSparse Matrix Collection were selected to cover diverse sparsity regimes and application domains, as summarized in Table I. Symmetric matrices are stored in expanded form to reflect raw CSR performance on the full non-zero structure.

III. RESULTS

Table III reports the throughput of every kernel on every matrix. Run-to-run variability is negligible.

TABLE I
SPARSE MATRIX STATISTICS ON PER-ROW NON-ZERO COUNT. SYMMETRIC MATRICES ARE STORED IN EXPANDED FORM.

Matrix	Dim.	nnz	med	avg	std	max
ASIC_680ks	683K	2.3M	2	3.4	4.3	210
FullChip	3.0M	26.6M	6	8.9	1806.8	2.3M
Rucci1*	2.0M	7.8M	4	3.9	0.3	4
Si41Ge41H72	186K	15.0M	37	80.9	127.0	662
bone010	987K	71.7M	81	72.6	15.8	81
boyd2	466K	1.5M	3	3.2	194.9	93K
eu-2005	863K	19.2M	16	22.3	29.3	7.0K
ldoor	952K	46.5M	49	48.9	12.0	77
rajat31	4.7M	20.3M	5	4.3	1.1	1.3K
webbase-1M	1.0M	3.1M	2	3.1	25.4	4.7K

*Rectangular: 2.0M $\times$ 110K; Dim. shows row count.

TABLE II
ONE-TIME SETUP COST AND KERNEL TIME. PARSING IS EXCLUDED.

	Conv. + H2D (ms)					Kernel (ms)				
Matrix	Scal.	Vec.	Adap.	Part.	cuSP.	Scal.	Vec.	Adap.	Part.	cuSP.
bone010	761	696	728	750	721	1.39	1.36	1.22	1.34	1.00
web-1M	46	42	48	52	45	0.50	0.54	0.08	0.26	0.09
ASIC	32	30	34	37	31	0.07	0.37	0.06	0.14	0.08
ldoor	528	482	621	524	487	1.02	0.99	0.78	1.10	0.67
rajat31	250	234	289	270	237	0.34	3.01	0.38	0.53	0.42
Rucci1	128	119	141	180	121	0.12	1.37	0.13	0.22	0.18
FullChip	312	311	438	325	320	199.3	10.1	9.4	8.2	0.48
Si41	160	160	226	163	167	0.45	0.25	0.26	0.41	0.24
boyd2	15	15	19	17	16	6.38	0.60	0.41	0.36	0.05
eu-05	234	234	325	247	249	0.99	0.73	0.36	0.59	0.31

cuSPARSE dominates on 6 of 10 matrices and peaks at 143.5 GFLOP/s on `bone010`. Two custom kernels overtake it on the remaining four: CSR-Adaptive on `ASIC_680ks` and `webbase-1M`, while CSR-Scalar on `Rucci1` and `rajat31`. On aggregate, CSR-Adaptive is the strongest custom kernel (59.4 GFLOP/s geomean), followed by CSR-Partial-Overlap (40.0); CSR-Scalar and CSR-Vector arrive tied at 22.8.

Two severe collapses stand out in: CSR-Scalar drops to 0.3 and 0.5 GFLOP/s on `FullChip` and `boyd2`, respectively, the two matrices with the largest single-row *nnz* in the dataset (Table I). CSR-Adaptive collapses on the same pair, reaching only 5.7 and 7.4 GFLOP/s. CSR-Vector remains below 14 GFLOP/s on six matrices: the four with the shortest rows (`ASIC_680ks`, `webbase-1M`, `Rucci1`, `rajat31`, all with median per-row *nnz* ≤ 5), plus `FullChip` and `boyd2`. CSR-Partial-Overlap shows the narrowest performance spread of the four custom kernels (16.5$\times$ between its minimum and maximum, against 20.9$\times$ for CSR-Adaptive, 24.1$\times$ for CSR-Vector, and 491.8$\times$ for CSR-Scalar), though it never matches the peak throughput of the other three. cuSPARSE, by contrast, spans only 2.5$\times$, remaining above 56 GFLOP/s across the entire portfolio.

A. Pre-processing Cost

Table II shows that setup time is dominated by the common COO-to-CSR conversion and CSR host-to-device transfer, which all kernels pay. CSR-Adaptive and Partal also include the CPU row-block builder and transfer of block metadata, but

TABLE III

THROUGHPUT (GFLOP/S) FOR ALL FIVE KERNELS ACROSS THE TEN MATRICES. BOLD MARKS THE BEST KERNEL PER MATRIX; † MARKS CELLS WHERE THE FLOAT32 RESULT FAILED THE 10^{-3} RELATIVE TOLERANCE. THE LAST COLUMN REPORTS THE GEOMETRIC MEAN ACROSS THE PORTFOLIO. STANDARD DEVIATION EXCEEDS 1 GFLOP/S ON FIVE PAIRS: CSR-PARTIAL-OVERLAP ON BONE010 (±9.5), CSR-VECTOR ON LDOOR (±3.8), CSR-SCALAR ON SI41GE41H72 (±1.8), BONE010 (±1.3), AND CSR-VECTOR ON RAJAT31 (±1.1); ALL OTHER PAIRS ARE BELOW ±1 GFLOP/S.

Kernel	bone010	web-1M	ASIC	ldoor	raj31	Rucci1	FChip	Si41	boyd2	eu-05	*Geo.*
CSR-Scalar	102.9†	12.4	64.7	91.1†	**119.7**	**131.4**	0.3	66.3	0.5	38.8	22.8
CSR-Vector	105.3†	11.6	12.6	93.6†	13.5	11.3	5.3	121.7	5.0	52.9	22.8
CSR-Adaptive	117.9	**80.4**	**74.3**	118.9	105.9	116.8	5.7	116.5	7.4	107.8	59.4
CSR-Partial-Overlap	107.4†	24.1	33.0	84.5†	77.4	72.6	6.5	73.1	8.3	65.4	40.0
cuSPARSE	**143.5**†	68.7	61.8	**139.0**†	96.3	85.6	**111.4**	**126.6**	**56.4**	**124.2**	**96.4**

this extra cost is small compared with the shared conversion cost. Therefore, similar setup times across kernels do not imply that partitioning is free; rather, its overhead is hidden inside a much larger common preprocessing cost.

IV. DISCUSSION

Each entry in Table III reflects a specific interaction between a kernel's memory-access strategy, its thread-work distribution, and the structural properties of the matrix.

A. Memory Coalescing and Access Patterns

Given SpMV's low arithmetic intensity, the results are explained mainly by memory-access efficiency and thread utilization rather than by raw floating-point throughput.

CSR-Scalar assigns one thread per row, so within a warp the i-th thread begins reading at `values[row_ptr[i]]`. For irregular row lengths, adjacent threads access non-contiguous addresses, producing 32 distinct cache-line requests per warp step instead of one. The gather `x[col_idx[j]]` adds a second source of irregularity through non-sequential column indices. Despite this, CSR-Scalar attains 77.8% of peak on `Rucci1` (maximum $nnz = 4$). With every row exactly 4 elements long, the 32 threads of a warp access `values[0..3]`, `values[4..7]`, ..., `values[124..127]`: a perfectly strided pattern that maps to a single cache-line transaction, accidentally recovering full coalescing. The same logic explains the 69.1% peak on `rajat31` (median $nnz = 5$). This uniform-short-row regime is the one structural niche where CSR-Scalar is competitive even against cuSPARSE.

CSR-Vector assigns one warp per row. All 32 lanes stride over `values[s], values[s+1], ...` with unit step, so reads of `values[]` and `col_idx[]` are fully coalesced regardless of row length. The bandwidth ceiling for this kernel is therefore determined almost entirely by the `x[col_idx]` scatter gather, which is inherently irregular. Where rows are long enough to keep all 32 lanes busy (`Si41Ge41H72`, median $nnz = 37$), CSR-Vector reaches 53.1% of peak, essentially matching CSR-Adaptive and approaching cuSPARSE. Where rows are short, warp underutilisation collapses bandwidth utilisation to under 8% (`ASIC_680ks`, `Rucci1`, `rajat31`).

CSR-Adaptive loads up to `NNZ_PER_BLOCK` = 1024 consecutive non-zeros cooperatively: all 256 threads issue reads separated by `THREADS_PER_BLOCK` = 256 elements, producing fully coalesced 256-wide transactions to `values[]` and `col_idx[]`. Together these account for the consistently high bandwidth utilisation (50–57%) across matrices with varied row-length distributions.

CSR-Partial-Overlap issues the same coalesced loads as CSR-Adaptive but pipelines them with `memcpy_async`, allowing the DMA engine to prefetch the next batch while the current one is being consumed from shared memory. However, the kernel's computation path caps `vector_size` at 32, meaning at most one warp is ever assigned to a single row. For matrices with very long rows, such as `FullChip` (max nnz = 2.3×10^6), those 32 threads must iterate ∼71,875 times per row while the rest of the block remains idle, a ceiling identical in effect to the single-warp fallback of the paper version of CSR-Adaptive. Rows that exceed `max_batch_size` are routed to the dedicated long-row kernel, which allocates a full block per row, but this path incurs additional kernel-launch overhead and does not benefit from the `memcpy_async` pipeline. Moreover for extremely-long rows, as in `FullChip`, this is insufficient since other methods use multiple thread blocks.

B. Load Imbalance and Warp Utilisation

Even when memory accesses are coalesced, a kernel wastes bandwidth if threads within a warp complete at different times. Two failure modes appear prominently.

Long-row serialisation defeats CSR-Scalar, CSR-Partial-Overlap, and CSR-Adaptive on `FullChip` and `boyd2`. On `FullChip`'s 2.3×10^6-nnz row, the CSR-Scalar thread assigned to it executes alone while 31 warp members wait, dropping effective HBM utilisation to 0.1% of peak. The paper version of CSR-Adaptive routes the same row to its single-warp CSR-Vector fallback (no row may exceed `NNZ_PER_BLOCK` without a dedicated long-row handler), reaching only 2.8% of peak.

Short-row warp underutilisation defeats CSR-Vector on the four matrices with median $nnz \leq 5$. On `ASIC_680ks` (median $nnz = 2$), each warp contributes on average 2 active lanes out of 32, for a warp occupancy of 6.3%, driving effective bandwidth to 7.8% of peak. CSR-Adaptive's Stream path avoids this by grouping multiple short rows into one block and assigning the entire block to a cooperative reduction, keeping all 256 threads busy regardless of individual row length.

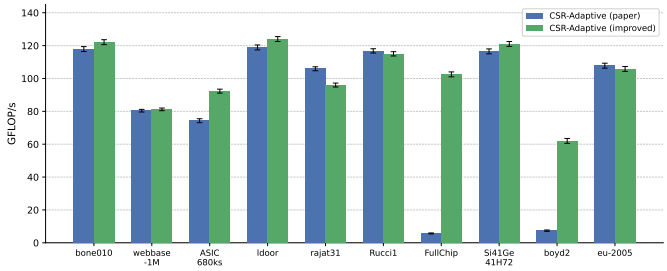

Fig. 1. Ablation of CSR-Adaptive: throughput (top) and speedup of the improved version over the paper version (bottom, log scale). Green bars indicate gain; red bars indicate regression.

C. Ablation: CSR-Adaptive Paper vs. Improved

Fig. 1 compares the paper-faithful implementation [4] against the improved version introduced in this work. The paper version's collapse on `FullChip` and `boyd2` traces to a missing case in the block builder: any row whose *nnz* exceeds `NNZ_PER_BLOCK` is routed to the single-warp CSR-Vector fallback discussed in Section IV-B. The improved version introduces a dedicated `csr_longrow_kernel` that splits such rows into 1024-element chunks, each processed by an independent thread block with partial sums accumulated via `atomicAdd`. The gains are $18\times$ on `FullChip` ($5.7 \rightarrow 102.3$ GFLOP/s) and $8.4\times$ on `boyd2` ($7.4 \rightarrow 62.1$ GFLOP/s); other matrices are either unaffected or show a small drop from the blocksize tradeoff.

D. Kernel-Level Profiling with Nsight Systems

Per-kernel timings on `FullChip`, whose 2.3×10^6-nnz long row stresses every long-row handler, corroborate the load-imbalance analysis of Sections IV-B and IV-C.

In CSR-Partial-Overlap, the long-row kernel consumes 7.2 ms (93.5% of total time) while the pipelined batch kernel completes in $505\,\mu$s, explaining why prefetching only pays off on matrices whose maximum row fits within a batch. The paper-faithful CSR-Adaptive is worse: the CSR-Vector fallback spends 8.0 ms on the long row against $360\,\mu$s of CSR-Stream for the remaining $\sim 26.6 \times 10^6$ non-zeros, so one row accounts for $\approx 96\%$ of kernel time. The improved version cuts the long-row contribution to $80\,\mu$s via block-level parallelisation ($400\,\mu$s Stream); the $480\,\mu$s aggregate is a $17.4\times$ reduction over the 8.36 ms paper baseline, matching the $18\times$ end-to-end speedup in Fig. 1 and confirming block-level parallelisation as the decisive factor on extreme row-length outliers.

E. Nsight Compute Comparison: CSR-Scalar vs. CSR-Vector

Table IV confirms that the two kernels fail for opposite reasons depending on matrix structure. On `Rucci1`, uniform 4-element rows let CSR-Scalar access contiguous memory across a warp, reaching 683.3 GB/s (73% of A30 peak) with 28.7 active threads. CSR-Vector wastes 28 of 32 lanes per row, collapsing to 57.4 GB/s and dropping GFLOP/s from 131.4 to 11.3. On `ASIC_680ks` the picture is similar but weaker:

TABLE IV
NSIGHT COMPUTE METRICS FOR CSR-SCALAR AND CSR-VECTOR. BW: DRAM THROUGHPUT (GB/S); DRAM%: FRACTION OF DEVICE PEAK; L1/L2: CACHE HIT RATES; THR: ACTIVE THREADS PER WARP; SC: WARP CYCLES PER ISSUED INSTRUCTION.

Matrix	Kernel	BW (GB/s)	DRAM (%)	L1 (%)	L2 (%)	Thr warp
ASIC_680ks	Scalar	307.9	33.0	78.7	37.0	14.3
	Vector	54.6	5.9	74.7	62.4	16.3
FullChip	Scalar	1.1	0.1	82.0	49.7	11.5
	Vector	23.1	2.5	64.6	56.4	18.1
Rucci1	Scalar	683.3	73.3	75.2	38.6	28.7
	Vector	57.4	6.2	78.5	66.8	16.5

CSR-Scalar still outperforms (307.9 vs. 54.6 GB/s) because short, nearly contiguous rows partially coalesce, while CSR-Vector again starves its warps. On `FullChip` the bottleneck shifts entirely to the 2.3M-non-zero row: CSR-Scalar serialises it onto one thread, yielding 1.1 GB/s and a 272 ms runtime (the 82% L1 hit rate reflects one thread reusing cached vector entries, not efficiency); CSR-Vector distributes the row across 32 threads ($22\times$ faster, 12.4 ms) but at 2.5% DRAM utilisation remains far from memory-bound.

V. CONCLUSION

The results confirm that CSR SpMV performance is determined less by raw floating-point throughput than by memory access efficiency and load balance. cuSPARSE is the most robust implementation overall, but custom kernels remain competitive on specific sparsity regimes. CSR-Scalar benefits from uniformly short rows, CSR-Vector from rows long enough to fill a warp, and CSR-Adaptive from grouping rows to reduce underutilisation. The main limitation is the treatment of extreme long rows, where splitting work across multiple blocks is essential to avoid serialization. Finally, all kernels leave the irregular gather `x[col_idx[j]]` unaddressed. Techniques that could reduce its randomness were left out of scope.

REFERENCES

[1] N. Bell and M. Garland, "Implementing sparse matrix-vector multiplication on throughput-oriented processors," in *Proc. Conf. High Performance Computing Networking, Storage and Analysis (SC '09)*, 2009, pp. 1–11.

[2] J. Gao, B. Liu, W. Ji, and H. Huang, "A systematic literature survey of sparse matrix-vector multiplication," 2024, arXiv:2404.06047. [Online]. Available: https://arxiv.org/abs/2404.06047

[3] G. Chu, Y. He, L. Dong, Z. Ding, D. Chen, H. Bai, X. Wang, and C. Hu, "Efficient algorithm design of optimizing SpMV on GPU," in *Proc. 32nd Int. Symp. High-Performance Parallel and Distributed Computing (HPDC '23)*, 2023, pp. 115–128. DOI: 10.1145/3588195.3593002.

[4] J. L. Greathouse and M. Daga, "Efficient sparse matrix-vector multiplication on GPUs using the CSR storage format," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC '14)*, IEEE, 2014, pp. 769–780.

[5] G. Zeng and Y. Zou, "Leveraging memory copy overlap for efficient sparse matrix-vector multiplication on GPUs," *Electronics*, vol. 12, no. 17, p. 3687, 2023. DOI: 10.3390/electronics12173687.